

# SMART LENDER – APPLICANT PREDICTION FOR LOAN APPROVAL

- PROJECT DOCUMENTATION

## **Submitted By**

- P. Appa Rao
- B.jyotsna
- T.Gnana Ahalladhini

## **DEPARTMENT:**

Computer Science and Engineering (Artificial Intelligence & Machine Learning

## **College**

GIET Engineering College,Rajamundry

## **Academic Year**

2025–2026



**SMARTBRIDGE**  
Let's Bridge the Gap

**SKILL  
WALLET**  
A SmartBridge Product

## ACKNOWLEDGEMENT:

We express our sincere gratitude to the management of **GIET Engineering College** for providing us with the opportunity to complete this project successfully.

We sincerely thank our Head of the Department, project guide, faculty members, and all teaching staff for their valuable guidance, encouragement, and continuous support throughout the project.

We would also like to thank **Smart Bridge** and **IBM Skills Build** for providing learning resources and project guidance.

Finally, we express our heartfelt gratitude to our parents, friends, and teammates for their encouragement and support throughout the development of this project.

## ABSTRACT

The **Smart Lender – Applicant Creditworthiness Prediction for Loan Approval** project is a Machine Learning-based web application developed to predict whether a loan applicant is eligible for loan approval.

Banks receive thousands of loan applications every day. Evaluating every application manually is time-consuming and may lead to errors. This project automates the loan approval prediction process using historical loan applicant data and Machine Learning algorithms.

The system analyzes important applicant details such as gender, marital status, education, income, loan amount, credit history, property area, and employment status to determine whether a loan should be approved or rejected.

Python is used for data preprocessing, model development, and prediction. Flask is used to build the web application interface through which users can enter applicant details and receive instant predictions.

The trained Machine Learning model achieved an accuracy of approximately **81.25%**, making it suitable for demonstrating intelligent loan prediction systems.

This project helps financial institutions reduce manual work, improve decision-making speed, and increase consistency in loan approval processes.

## **TABLE OF CONTENTS**

1. Introduction
2. Problem Statement
3. Objectives
4. Existing System
5. Proposed System
6. Software Requirements
7. Hardware Requirements
8. Project Flow
9. System Architecture
10. Dataset Description
11. Epic 1 – Data Collection and Architecture Design
12. Epic 2 – Visualizing and Analysing the Data
13. Epic 3 – Data Pre-processing
14. Epic 4 – Model Building
15. Epic 5 – Application Building
16. Testing
17. Results
18. Advantages
19. Limitations
20. Future Enhancements
21. Conclusion
22. References

## INTRODUCTION

Machine Learning has become one of the most important technologies in the banking and financial sectors. It enables organizations to analyze customer data, identify patterns, and make intelligent decisions with minimal human intervention.

Loan approval is one of the most critical tasks performed by banks and financial institutions. Traditionally, bank officers verify applicant information manually before approving loans. This manual process requires significant time and effort and may result in inconsistent decisions.

The Smart Lender project addresses this challenge by developing an intelligent loan prediction system using Machine Learning techniques. The application predicts whether an applicant is eligible for a loan based on historical loan records and applicant information.

The project uses a dataset containing **614 loan applications** with **13 attributes**, including applicant income, co-applicant income, loan amount, education, employment status, property area, and credit history.

The data is preprocessed, analyzed, and used to train a Machine Learning model. The trained model is then integrated into a Flask web application where users can enter applicant details and instantly receive loan approval predictions.

The Smart Lender application demonstrates how Machine Learning can support banks in improving efficiency, reducing processing time, and assisting loan officers in making data-driven decisions.

## PROBLEM STATEMENT

Banks and financial institutions receive a large number of loan applications every day. Before approving a loan, the bank verifies several factors such as applicant income, employment status, education, credit history, loan amount, and repayment capability.

The traditional loan approval process is mostly manual, which consumes a significant amount of time and resources. Human evaluation may also lead to inconsistencies and errors in decision-making. As the number of loan applications increases, it becomes difficult for banks to process every application quickly while maintaining accuracy.

To overcome these challenges, there is a need for an intelligent system that can analyze applicant details automatically and predict whether the applicant is eligible for a loan. Such a system can support bank officials by providing fast and reliable recommendations based on historical loan data.

The **Smart Lender** project addresses this problem by using Machine Learning techniques to predict loan approval decisions based on applicant information. This helps improve efficiency, reduce manual effort, and enhance the overall loan approval process.

## OBJECTIVES

The main objectives of the Smart Lender project are:

- To develop an intelligent loan approval prediction system using Machine Learning.
- To analyze historical loan applicant data for identifying approval patterns.
- To preprocess and clean the dataset for accurate model training.
- To build a Machine Learning model capable of predicting loan approval.
- To create a user-friendly web application using Flask.
- To provide instant loan approval predictions based on user inputs.
- To reduce manual effort in the loan evaluation process.
- To improve the speed and consistency of loan approval decisions.
- To demonstrate the practical application of Machine Learning in the banking sector

## **EXISTING SYSTEM**

In the existing loan approval system, loan applications are verified manually by bank employees. Each applicant's information is reviewed carefully before making a final decision.

The approval process depends on several factors such as income, employment status, education, loan amount, and repayment history. Since this evaluation is performed manually, it requires considerable time and effort.

### **Limitations of the Existing System**

- Manual verification consumes a significant amount of time.
- Human errors may occur during evaluation.
- Decision-making may vary between different employees.
- Processing a large number of applications becomes difficult.
- Increased operational costs.
- Slower customer service.
- Difficulty in identifying hidden patterns in applicant data.

These limitations highlight the need for an automated loan prediction system.

## **PROPOSED SYSTEM**

The proposed system introduces a Machine Learning-based loan approval prediction model that automates the loan evaluation process.

The application uses historical loan records to train a prediction model. Applicant information such as gender, marital status, income, education, employment status, loan amount, credit history, and property area is analyzed to predict loan approval.

A Flask web application provides an easy-to-use interface where users can enter applicant details. The trained Machine Learning model processes these inputs and predicts whether the loan is likely to be approved or rejected.

### **Benefits of the Proposed System**

- Faster loan approval prediction.
- Reduced manual work.
- Improved prediction accuracy.

- Consistent decision-making.
- Easy-to-use web interface.
- Supports banking professionals in making better decisions.
- Can be extended with additional features in the future.

## SOFTWARE REQUIREMENTS

The following software tools were used during the development of the Smart Lender project.

| Software                  | Purpose                          |
|---------------------------|----------------------------------|
| Python 3.x                | Programming Language             |
| Jupyter Notebook          | Data Analysis and Model Training |
| Flask                     | Web Application Development      |
| HTML                      | User Interface                   |
| CSS                       | Web Page Styling                 |
| Pandas                    | Data Manipulation                |
| NumPy                     | Numerical Computation            |
| Matplotlib                | Data Visualization               |
| Seaborn                   | Statistical Visualization        |
| Scikit-learn              | Machine Learning Algorithms      |
| Pickle                    | Model Serialization              |
| Visual Studio Code / IDLE | Code Development                 |
| GitHub                    | Version Control                  |

## HARDWARE REQUIREMENTS

| Hardware                                                         | Specification           |
|------------------------------------------------------------------|-------------------------|
| Processor                                                        | Intel Core i3 or Higher |
| RAM                                                              | Minimum 4 GB            |
| Hard Disk                                                        | 20 GB Free Space        |
| Operating System                                                 | Windows 10/11           |
| Internet Connection Required for GitHub and Package Installation |                         |

## PRE-REQUISITES

Before starting the project, the following prerequisites were completed:

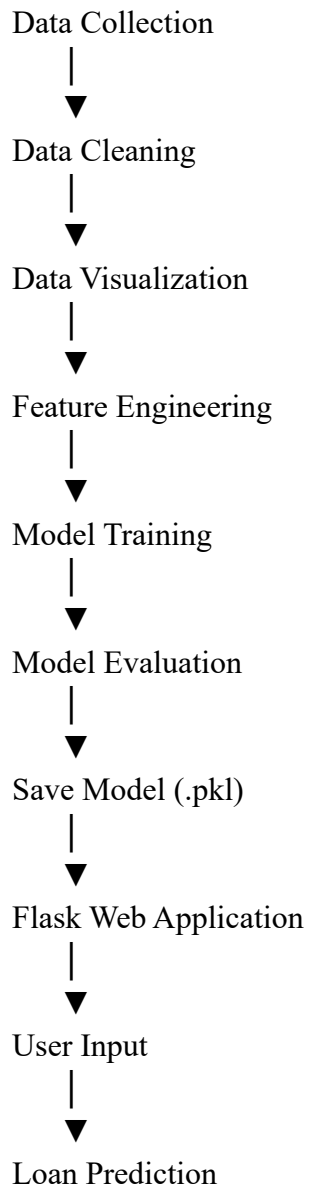
- Python installation
- Jupyter Notebook setup
- Flask installation
- Required Python libraries installation
- Dataset collection
- GitHub repository creation
- HTML template preparation
- Machine Learning environment setup
- Model training environment
- Project folder organization

## PROJECT FLOW

The Smart Lender project follows a structured workflow from data collection to final prediction.

Loan Dataset





### **Explanation of the Workflow**

1. The dataset is collected from the loan records.
2. Missing values and inconsistent data are cleaned.
3. Data visualization is performed to understand patterns.
4. Categorical values are encoded into numerical values.
5. The dataset is divided into training and testing sets.

6. A Machine Learning model is trained.
7. The trained model is evaluated and saved as a .pkl file.
8. Flask loads the trained model.
9. The user enters applicant details.
10. The application predicts whether the loan will be **Approved** or **Rejected**.

## EPIC 1 – DATA COLLECTION AND ARCHITECTURE DESIGN

### Overview

Data collection is the first and one of the most important phases in any Machine Learning project. The quality of the dataset directly affects the performance of the prediction model. In the Smart Lender project, a loan applicant dataset containing historical loan application records was used for model training and testing.

The dataset contains information about applicants such as their gender, marital status, education, employment status, income, loan amount, credit history, and loan approval status. Using this information, the Machine Learning model learns patterns that help predict whether a future applicant is likely to receive loan approval.

### Dataset Description

The Smart Lender project uses a loan applicant dataset named **train.csv**.

### Dataset Information

| Attribute | Value |
|-----------|-------|
|-----------|-------|

|              |           |
|--------------|-----------|
| Dataset Name | train.csv |
|--------------|-----------|

|               |     |
|---------------|-----|
| Total Records | 614 |
|---------------|-----|

|               |    |
|---------------|----|
| Total Columns | 13 |
|---------------|----|

|           |                        |
|-----------|------------------------|
| Data Type | Structured CSV Dataset |
|-----------|------------------------|

|                 |             |
|-----------------|-------------|
| Target Variable | Loan_Status |
|-----------------|-------------|

The dataset contains both categorical and numerical features. The target column **Loan\_Status** indicates whether a loan application was approved (Y) or rejected (N).

### Dataset Attributes

The dataset contains the following attributes:

| Column Name | Description |
|-------------|-------------|
|-------------|-------------|

|         |                                   |
|---------|-----------------------------------|
| Loan_ID | Unique Loan Identification Number |
|---------|-----------------------------------|

| Column Name       | Description                    |
|-------------------|--------------------------------|
| Gender            | Applicant Gender (Male/Female) |
| Married           | Marital Status                 |
| Dependents        | Number of Dependents           |
| Education         | Graduate or Not Graduate       |
| Self_Employed     | Employment Status              |
| ApplicantIncome   | Monthly Income of Applicant    |
| CoapplicantIncome | Monthly Income of Co-applicant |
| LoanAmount        | Requested Loan Amount          |
| Loan_Amount_Term  | Loan Repayment Duration        |
| Credit_History    | Previous Credit Record         |
| Property_Area     | Urban / Semiurban / Rural      |
| Loan_Status       | Loan Approval Status (Y/N)     |

### Sample Dataset

| Loan_ID  | Gender | Married | Income | Loan Amount | Credit History | Loan Status |
|----------|--------|---------|--------|-------------|----------------|-------------|
| LP001002 | Male   | No      | 5849   | NA          | 1              | Y           |
| LP001003 | Male   | Yes     | 4583   | 128         | 1              | N           |
| LP001005 | Male   | Yes     | 3000   | 66          | 1              | Y           |
| LP001006 | Male   | Yes     | 2583   | 120         | 1              | Y           |
| LP001008 | Male   | No      | 6000   | 141         | 1              | Y           |

This sample shows how applicant information is stored in the dataset and how the corresponding loan approval status is represented.

### **Dataset Information (info())**

The dataset contains **614 entries** and **13 columns**.

### **Data Types**

#### **Data Type Count**

Object      8

Integer     1

Float       4

Some columns contain missing values that need preprocessing before model training.

Examples include:

- Gender
- Married
- Dependents
- Self\_Employed
- LoanAmount
- Loan\_Amount\_Term
- Credit\_History

These missing values are handled during the data preprocessing stage.

### **Statistical Analysis (describe())**

The numerical columns were analyzed using the describe() function.

#### **Summary Statistics**

| <b>Feature</b>      | <b>Mean</b> | <b>Minimum</b> | <b>Maximum</b> |
|---------------------|-------------|----------------|----------------|
| Applicant Income    | 5403.46     | 150            | 81000          |
| Co-applicant Income | 1621.25     | 0              | 41667          |
| Loan Amount         | 146.41      | 9              | 700            |

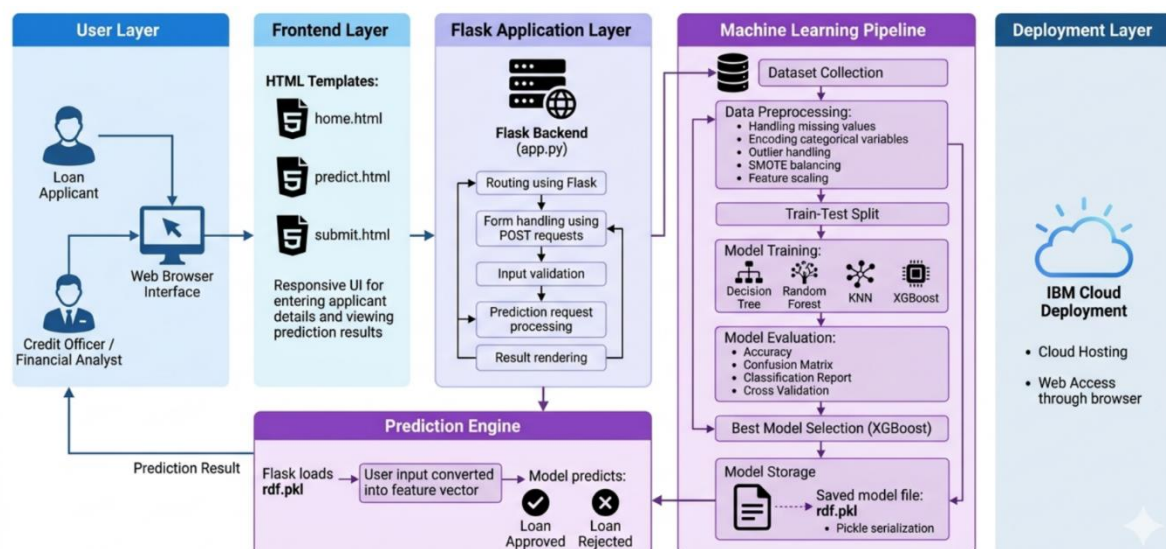
| Feature          | Mean       | Minimum | Maximum |
|------------------|------------|---------|---------|
| Loan Amount Term | 342 Months | 12      | 480     |
| Credit History   | 0.842      | 0       | 1       |

### Interpretation

- The average applicant income is **₹5403.46**.
- Most loan repayment terms are **360 months**.
- Credit history values are mostly **1**, indicating many applicants have a good credit record.
- Loan amounts vary significantly from **9** to **700**, showing the need for normalization and careful model training.

### System Architecture

The Smart Lender system is designed using a Machine Learning workflow integrated with a Flask web application.



### Architecture Explanation

The architecture consists of the following components:

1. **Loan Dataset** – Historical loan application records.
2. **Data Preprocessing** – Missing value handling and feature encoding.

3. **Machine Learning Model** – Trained using processed data.
4. **Model File (loan\_model.pkl)** – Stores the trained model.
5. **Flask Web Application** – Connects the user interface with the model.
6. **User Interface** – Accepts applicant details from users.
7. **Prediction Engine** – Predicts loan approval or rejection.
8. **Result Page** – Displays the prediction result.

This architecture ensures a smooth flow from user input to final prediction.

### **Benefits of Data Collection**

The collected dataset provides several advantages:

- Covers both categorical and numerical information.
- Represents real-world loan application scenarios.
- Suitable for supervised Machine Learning.
- Helps the model learn meaningful approval patterns.
- Improves prediction performance when properly preprocessed.

### **Summary**

In this phase, the loan dataset was collected and analyzed. The dataset structure, attributes, statistical properties, and system architecture were studied. This forms the foundation for the next stages of visualization, preprocessing, and model building.

## **EPIC 2 – VISUALIZING AND ANALYSING THE DATA**

### **Overview**

Data visualization is one of the most important phases in a Machine Learning project. It helps us understand the structure of the dataset, identify hidden patterns, detect missing values, observe relationships between variables, and make informed decisions before model training.

In the Smart Lender project, different visualization techniques were used to analyze applicant information such as gender, education, income, property area, loan amount, and credit history. The insights obtained from these visualizations helped improve data preprocessing and model performance.

### **Objectives of Data Visualization**

The objectives of data visualization in this project are:

- To understand the distribution of applicant information.
- To identify missing values in the dataset.
- To study the relationship between independent variables and loan approval.
- To detect outliers in numerical columns.
- To understand applicant income patterns.
- To analyze loan approval trends.
- To improve the quality of data before model training.

### **Distribution of Loan Status**

#### **Observation**

The Loan Status column contains two categories:

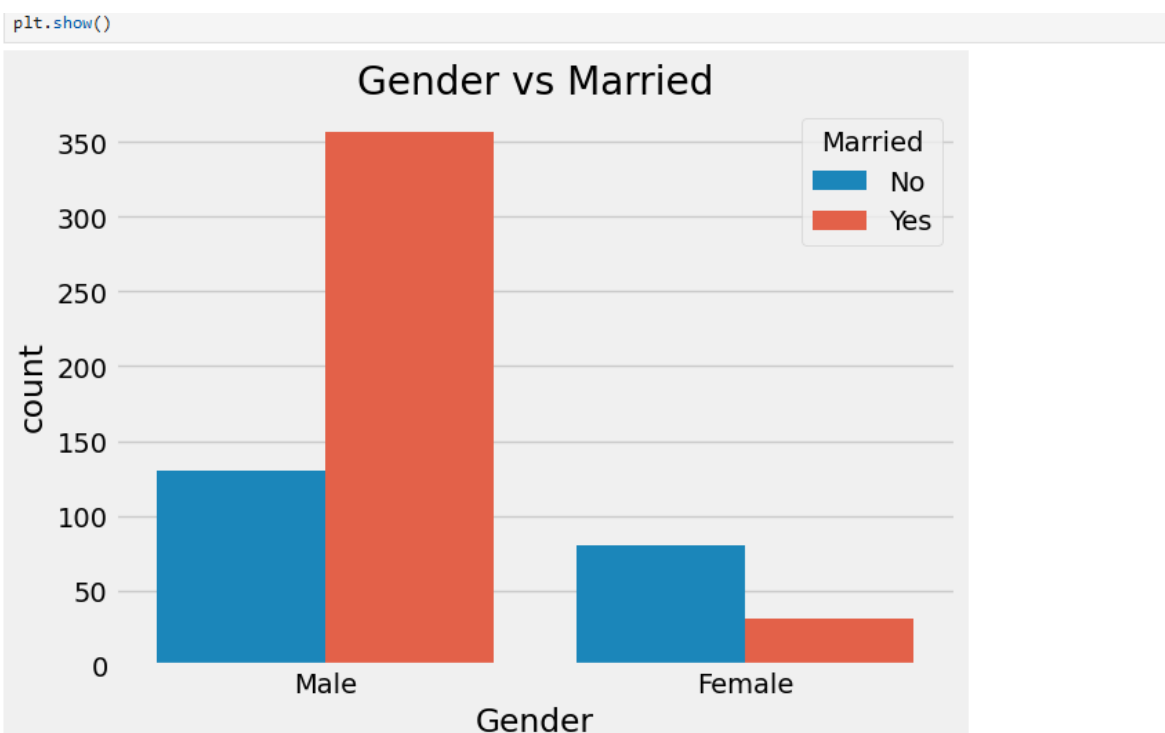
- Y – Loan Approved
- N – Loan Rejected

From the visualization, it can be observed that the number of approved loans is higher than rejected loans. This indicates that most applicants in the historical dataset were eligible for loan approval.

## Interpretation

This imbalance is common in banking datasets because financial institutions generally receive applications from customers who already satisfy many eligibility criteria.

## Gender Distribution



## Observation

The Gender feature contains:

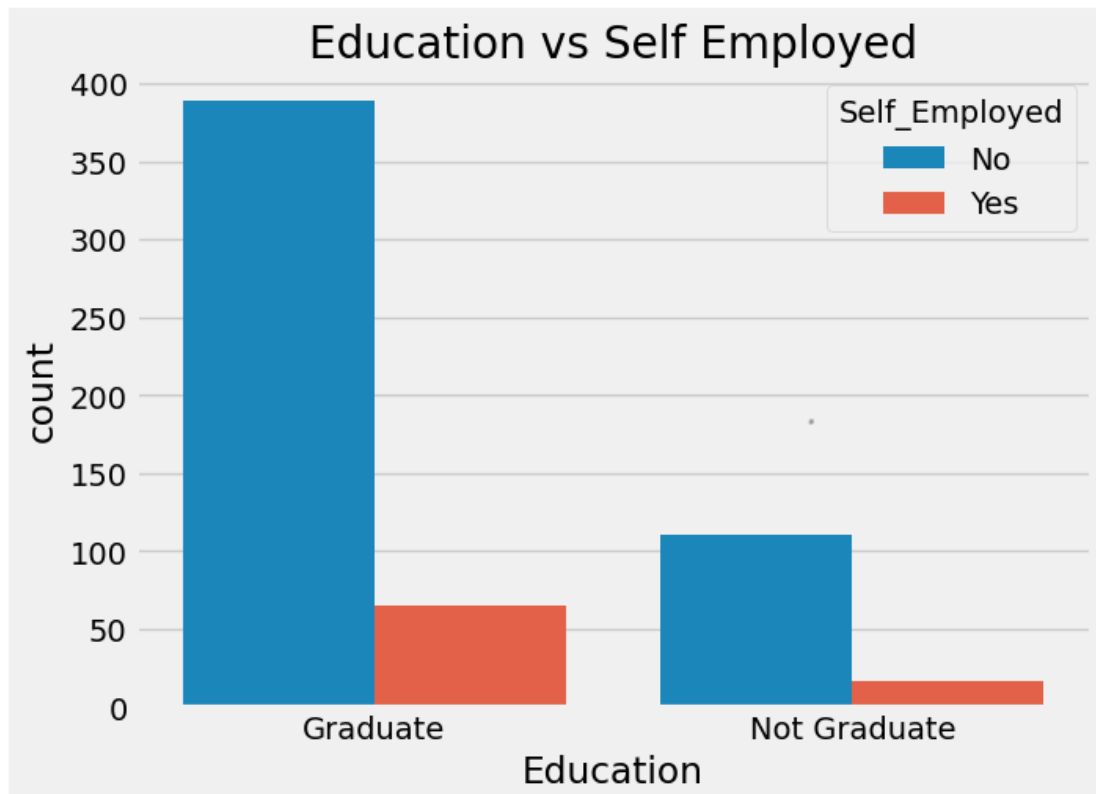
- Male
- Female

The dataset contains more male applicants than female applicants.

## Interpretation

Gender is considered one of the categorical features in the dataset. Although it is not the primary deciding factor, it contributes to the prediction process along with other features.

## Observation



Applicants are classified into:

- Graduate
- Not Graduate

The visualization shows that most applicants are graduates.

### Interpretation

Educational qualification may influence employment opportunities and income level, indirectly affecting loan approval decisions.

### Self-Employment Analysis

#### Observation

Applicants are divided into:

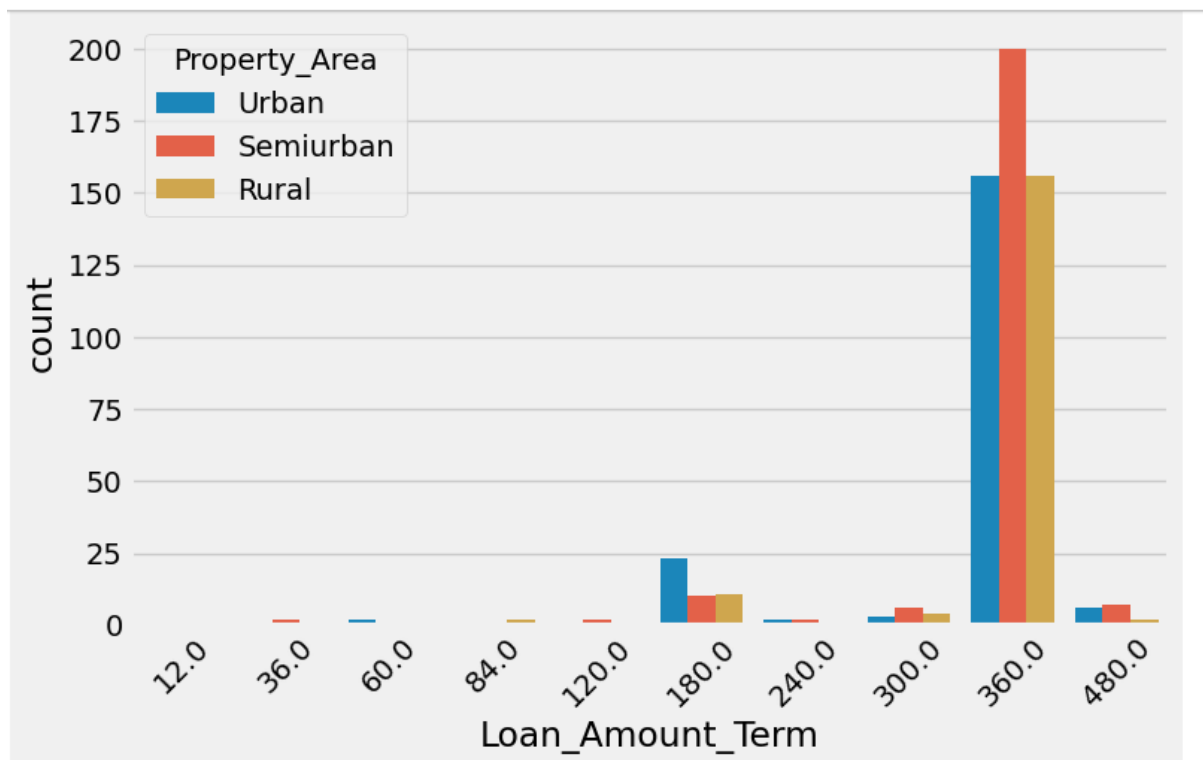
- Self Employed
- Not Self Employed

Most applicants belong to the non-self-employed category.

## Interpretation

Employment status provides useful information regarding income stability and repayment capability.

## Property Area Analysis



## Observation

Property areas include:

- Urban
- Semiurban
- Rural

Semiurban and Urban areas have a larger number of applicants compared to Rural areas.

## Interpretation

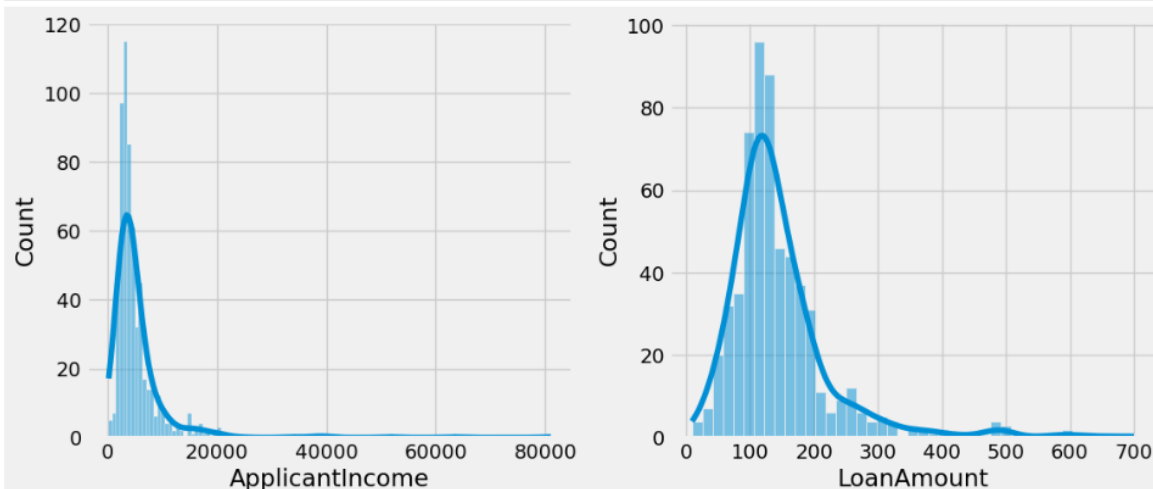
Property location may influence employment opportunities and financial stability, which can affect loan approval.

## Applicant Income Distribution

```
plt.subplot(1,2,1)
sns.histplot(df["ApplicantIncome"], kde=True)

plt.subplot(1,2,2)
sns.histplot(df["LoanAmount"], kde=True)

plt.show()
```



### Observation

Applicant income ranges from ₹150 to ₹81,000.

Most applicants have income below ₹10,000, while a few applicants have exceptionally high incomes.

### Interpretation

The income distribution is positively skewed because a small number of applicants earn significantly higher salaries than the majority.

## Co-applicant Income Distribution

### Observation

Many applicants have zero co-applicant income, while others have varying income levels.

### Interpretation

The co-applicant income increases the total household income and may improve the probability of loan approval.

## Loan Amount Distribution

### Observation

Loan amounts vary between **9** and **700**.

Most applicants request moderate loan amounts.

### **Interpretation**

Applicants requesting smaller loans generally have better approval chances compared to those requesting very large amounts.

## **Credit History Analysis**

### **Observation**

Credit History contains two values:

- **1** – Good Credit History
- **0** – Poor Credit History

Most applicants have a credit history value of **1**.

### **Interpretation**

Credit History is one of the strongest factors affecting loan approval. Applicants with a good credit history have a much higher probability of receiving loan approval.

## **Correlation Analysis**

### **Observation**

Correlation analysis helps identify relationships between numerical variables.

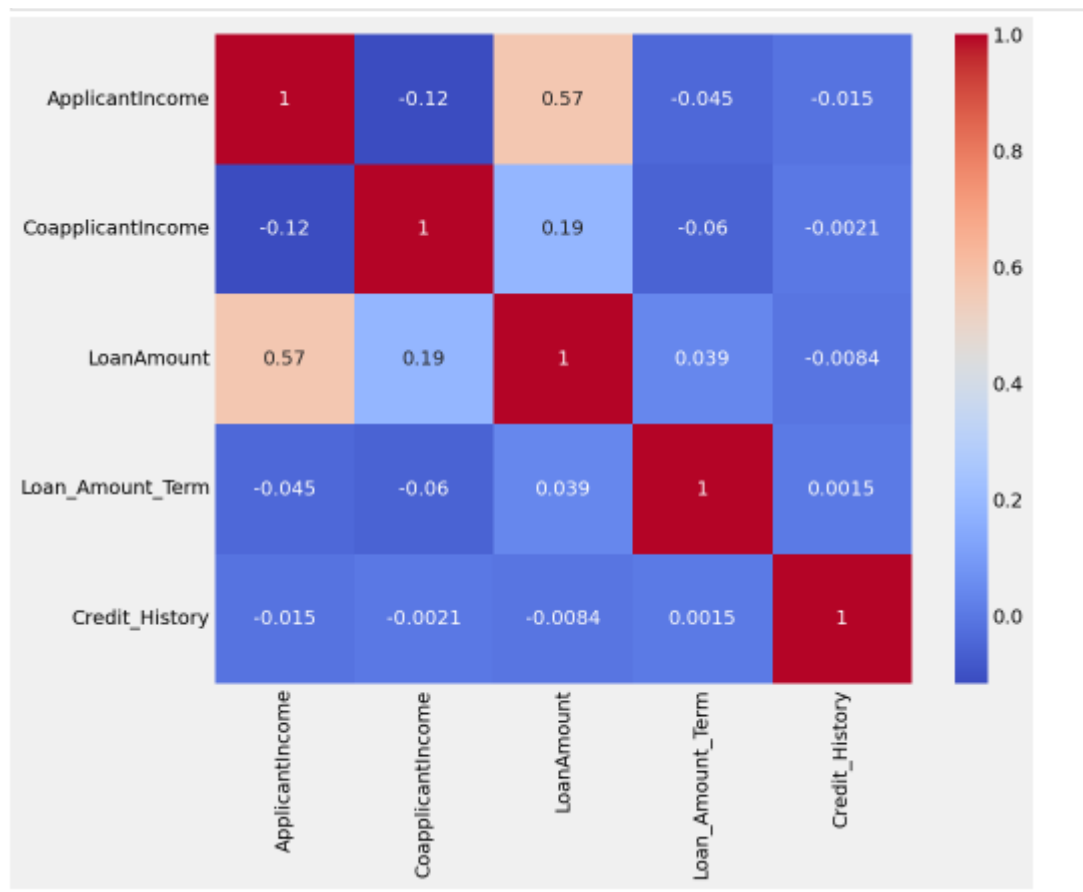
Important observations include:

- Applicant Income has a moderate relationship with Loan Amount.
- Credit History has a strong influence on Loan Status.
- Loan Amount and Loan Term show limited correlation.

### **Interpretation**

Understanding correlations helps in selecting important features for Machine Learning model training.

## **Outlier Detection**



### Observation

Outliers are observed in:

- Applicant Income
- Co-applicant Income
- Loan Amount

### Interpretation

Outliers can negatively affect Machine Learning performance. During preprocessing, these values are carefully handled to improve prediction accuracy.

### Key Insights

The following insights were obtained after analyzing the dataset:

- Most applicants are graduates.

- Male applicants are more common than female applicants.
- Most loan applications are approved.
- Good credit history significantly increases approval chances.
- Applicant income varies considerably.
- Most loans have a repayment period of **360 MONTHS**.
- Missing values exist in multiple columns and require preprocessing.
- Numerical features contain outliers that should be handled appropriately.

### Summary

In this phase, the loan dataset was explored using various visualization techniques. The analysis helped identify data distributions, missing values, feature relationships, and important factors influencing loan approval. These insights guided the preprocessing and model-building phases, ultimately contributing to better prediction accuracy.

## **EPIC 3 – DATA PRE-PROCESSING**

### **Overview**

Data preprocessing is one of the most important phases in Machine Learning. Real-world datasets often contain missing values, inconsistent data, duplicate records, and categorical variables that cannot be directly processed by Machine Learning algorithms.

The objective of preprocessing is to transform the raw dataset into a clean, structured, and machine-readable format. Proper preprocessing improves model performance, increases prediction accuracy, and minimizes errors during training.

In the Smart Lender project, several preprocessing techniques were applied to prepare the loan dataset for model development.

### **Need for Data Pre-processing**

The loan dataset contains:

- Missing values
- Categorical variables
- Numerical variables
- Different data types
- Outliers

Machine Learning algorithms cannot directly process text values such as "Male", "Female", "Graduate", or "Urban". Therefore, preprocessing converts these values into numerical representations.

### **Checking Missing Values**

The dataset was examined to identify missing values in each column.

### **Missing Value Summary**

| <b>Column</b> | <b>Missing Values</b> |
|---------------|-----------------------|
| Gender        | 13                    |

| Column           | Missing Values |
|------------------|----------------|
| Married          | 3              |
| Dependents       | 15             |
| Self_Employed    | 32             |
| LoanAmount       | 22             |
| Loan_Amount_Term | 14             |
| Credit_History   | 50             |

The remaining columns did not contain missing values.

### Observation

Missing values can reduce model performance if they are not handled correctly. Therefore, each missing value was replaced using suitable statistical techniques.

### Handling Missing Values

Different strategies were applied depending on the type of data.

#### Categorical Columns

For categorical attributes such as:

- Gender
- Married
- Dependents
- Self\_Employed

missing values were replaced using the **Mode (most frequently occurring value)**.

#### Numerical Columns

For numerical attributes such as:

- LoanAmount
- Loan\_Amount\_Term

missing values were replaced using the **Median**, which is less sensitive to outliers.

### **Credit History**

Missing values in Credit\_History were also replaced using the most frequent value because it is a categorical indicator (0 or 1).

### **Removing Unnecessary Columns**

The **Loan\_ID** column was removed before model training.

### **Reason**

Loan\_ID is only a unique identifier and does not contribute to predicting loan approval.

Removing unnecessary features helps reduce computational complexity and improves model efficiency.

### **Label Encoding**

Machine Learning models require numerical input. Therefore, categorical values were converted into numerical values using **Label Encoding**.

### **Encoded Values**

#### **Gender**

##### **Original Encoded**

Male     1

Female   0

#### **Married**

##### **Original Encoded**

Yes       1

No        0

#### **Education**

### **Original      Encoded**

Graduate      1

Not Graduate 0

### **Self Employed**

#### **Original Encoded**

Yes      1

No      0

### **Property Area**

#### **Original      Encoded**

Rural      0

Semiurban 1

Urban      2

### **Loan Status (Target Variable)**

#### **Original Encoded**

N      0

Y      1

### **Feature Selection**

The following features were selected for model training:

- Gender
- Married
- Dependents

- Education
- Self\_Employed
- ApplicantIncome
- CoapplicantIncome
- LoanAmount
- Loan\_Amount\_Term
- Credit\_History
- Property\_Area

### **Target Variable**

Loan\_Status

### **Splitting the Dataset**

The cleaned dataset was divided into two parts:

#### **Training Dataset**

- Used for learning patterns
- Approximately 80%

#### **Testing Dataset**

- Used for evaluating performance
- Approximately 20%

This approach helps evaluate how well the trained model performs on unseen data.

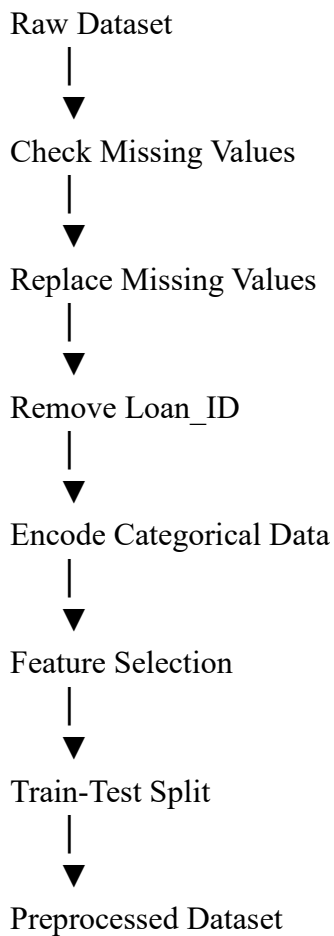
### **Advantages of Data Pre-processing**

Proper preprocessing offers several advantages:

- Improves data quality.
- Eliminates inconsistencies.
- Converts categorical data into numerical form.
- Handles missing values effectively.

- Reduces model errors.
- Improves prediction accuracy.
- Makes the dataset suitable for Machine Learning algorithms.

### **Pre-processing Workflow**



This workflow ensures that the data is clean, consistent, and ready for model training.

### **Epic 3 Summary**

In this phase, the loan dataset was cleaned and transformed into a machine-readable format. Missing values were handled, categorical variables were encoded, unnecessary columns were removed, and the dataset was split into training and testing sets. These preprocessing steps significantly improved the quality of the data and prepared it for building an accurate Machine Learning model.

## EPIC 4 – MODEL BUILDING

### Overview

Model Building is the core phase of the Smart Lender project. After completing data collection, visualization, and preprocessing, the cleaned dataset was used to train a Machine Learning model capable of predicting whether a loan application should be approved or rejected.

The model learns patterns from historical loan records and uses those patterns to make predictions for new applicants. A well-trained model reduces manual effort and provides quick and consistent loan approval decisions.

### Machine Learning Algorithm

A **Random Forest Classifier** was selected for this project.

### Why Random Forest?

The Random Forest algorithm combines multiple decision trees and produces predictions based on majority voting. It provides better accuracy and reduces overfitting compared to a single decision tree.

### Advantages

- High prediction accuracy.
- Handles both numerical and categorical features.
- Less prone to overfitting.
- Works well with missing and noisy data.
- Provides stable predictions.

### Training the Model

After preprocessing, the dataset was divided into:

| Dataset | Percentage |
|---------|------------|
|---------|------------|

|               |     |
|---------------|-----|
| Training Data | 80% |
|---------------|-----|

|              |     |
|--------------|-----|
| Testing Data | 20% |
|--------------|-----|

The training dataset was used to train the Random Forest model, while the testing dataset was used to evaluate its performance.

### Features Used for Training

The following features were used as input variables:

| Feature           | Description             |
|-------------------|-------------------------|
| Gender            | Applicant Gender        |
| Married           | Marital Status          |
| Dependents        | Number of Dependents    |
| Education         | Education Qualification |
| Self_Employed     | Employment Status       |
| ApplicantIncome   | Monthly Income          |
| CoapplicantIncome | Co-applicant Income     |
| LoanAmount        | Requested Loan Amount   |
| Loan_Amount_Term  | Loan Repayment Period   |
| Credit_History    | Previous Credit Record  |
| Property_Area     | Applicant Location      |

### Target Variable

#### Loan\_Status

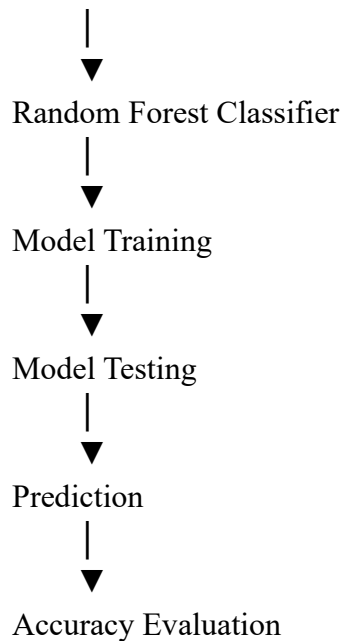
- Y → Loan Approved
- N → Loan Rejected

### Model Training Workflow

Preprocessed Dataset



Train-Test Split



This workflow shows how the Machine Learning model was trained and evaluated.

### **Model Accuracy**

After training, the model achieved the following accuracy:

**Accuracy = 81.25%**

### **Interpretation**

An accuracy of **81.25%** indicates that the model correctly predicts loan approval or rejection for approximately **81 out of every 100 applications** in the testing dataset.

This demonstrates that the model has learned meaningful patterns from historical data and can provide reliable predictions.

### **Model Evaluation**

The model was evaluated using the testing dataset.

### **Evaluation Steps**

1. Train the Random Forest model.
2. Predict loan status for testing data.
3. Compare predicted values with actual values.

#### 4. Calculate prediction accuracy.

This process ensures that the model performs well on unseen data and is suitable for deployment.

### **Saving the Model**

Once the model achieved satisfactory accuracy, it was saved using the **Pickle** library.

### **Saved File**

**loan\_model.pkl**

### **Purpose**

- Stores the trained model permanently.
- Eliminates the need to retrain the model every time.
- Can be directly loaded into the Flask application for predictions.

### **Prediction Process**

The prediction process follows these steps:

1. User enters applicant details.
2. Flask receives the input.
3. Input data is converted into numerical format.
4. The trained model processes the input.
5. The model predicts the loan status.
6. The result is displayed as:
  - **Loan Approved**
  - **Loan Rejected**

### **Example Prediction**

#### **Example 1**

| <b>Feature</b>   | <b>Value</b> |
|------------------|--------------|
| Applicant Income | ₹50,000      |

| Feature             | Value    |
|---------------------|----------|
| Co-applicant Income | ₹10,000  |
| Loan Amount         | ₹150     |
| Credit History      | 1        |
| Education           | Graduate |

#### Prediction

 **Loan Approved**

#### Example 2

| Feature             | Value        |
|---------------------|--------------|
| Applicant Income    | ₹8,000       |
| Co-applicant Income | ₹0           |
| Loan Amount         | ₹300         |
| Credit History      | 0            |
| Education           | Not Graduate |

#### Prediction

 **Loan Rejected**

#### Advantages of the Model

The trained model provides several benefits:

- Fast loan approval prediction.
- Consistent decision-making.
- Reduces manual effort.
- Learns from historical loan records.
- Suitable for real-time prediction through Flask.

- Achieves good prediction accuracy (81.25%).

### Summary

In this phase, a Random Forest Machine Learning model was trained using the preprocessed loan dataset. The model was evaluated using testing data and achieved an accuracy of **81.25%**. The trained model was then saved as **loan\_model.pkl** for integration with the Flask web application. This phase forms the intelligent decision-making component of the Smart Lender system.

## **EPIC 5 – APPLICATION BUILDING**

### **Overview**

After successfully training the Machine Learning model, the next phase involved developing a web application that allows users to interact with the trained model. The application was developed using the Flask framework, which provides a lightweight and efficient platform for deploying Machine Learning models.

The Smart Lender web application allows users to enter applicant details through a simple interface. The entered information is sent to the trained model, which predicts whether the loan application is likely to be approved or rejected.

### **Flask Framework**

Flask is a lightweight Python web framework used for building web applications.

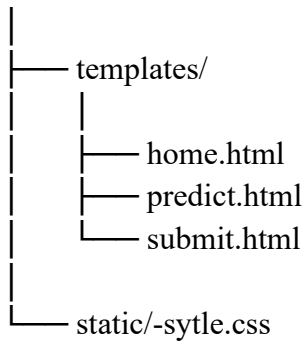
### **Advantages of Flask**

- Easy to develop web applications.
- Lightweight and fast.
- Integrates easily with Machine Learning models.
- Supports HTML templates.
- Simple routing mechanism.
- Easy deployment.

### **Project Folder Structure**

The project follows the structure shown below.

```
Smart-Lender/  
|  
|— app.py  
|— loan_model.pkl  
|— train.csv  
|— loan_model.ipynb
```



This structure separates the Machine Learning model, application code, templates, and dataset for better organization.

## Application Components

The Smart Lender application consists of the following major components.

### 1. app.py

The **app.py** file acts as the main controller of the application.

Functions performed:

- Creates Flask application.
- Loads the trained Machine Learning model.
- Receives user input.
- Sends data to the prediction model.
- Displays prediction results.

### 2. rdf.pkl

This file stores the trained Random Forest model.

Purpose:

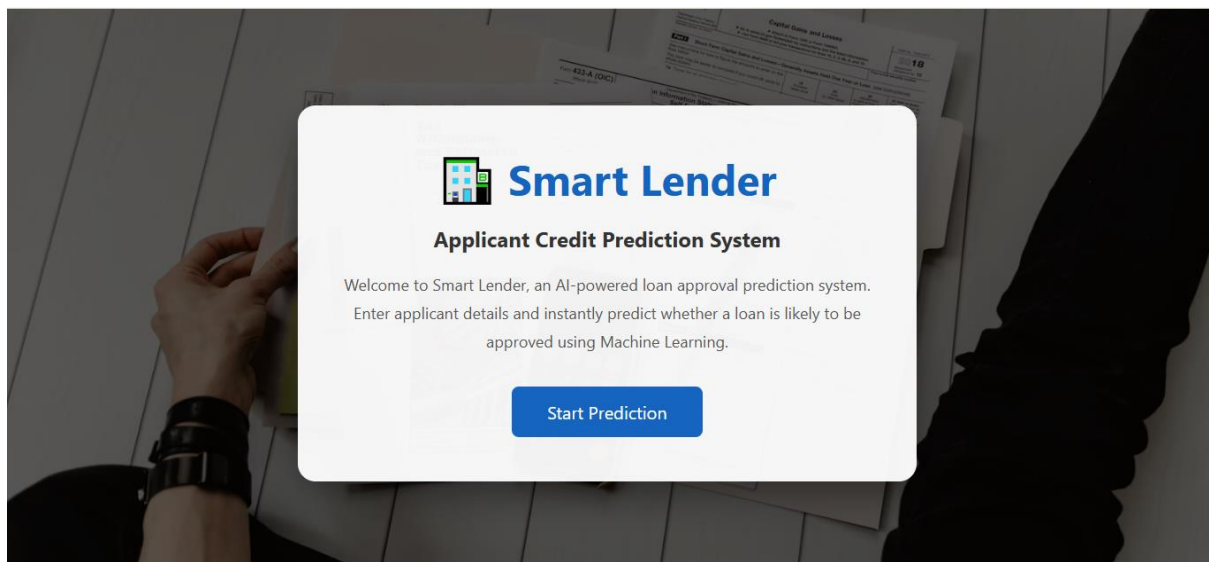
- Avoids retraining every time.
- Loads instantly during application startup.
- Performs real-time prediction.

### 3. home.html

This is the welcome page of the application.

Functions:

- Displays project title.
- Provides navigation.
- Redirects users to the prediction page.



### 4. predict.html

This page contains the loan application form.

The user enters:

- Gender
- Married
- Dependents
- Education
- Self Employed
- Applicant Income
- Co-applicant Income

- Loan Amount
- Loan Amount Term
- Credit History
- Property Area

|                                    |                                  |
|------------------------------------|----------------------------------|
| <b>Gender</b><br>Male              | <b>Married</b><br>Yes            |
| <b>Dependents</b><br>2             | <b>Education</b><br>Graduate     |
| <b>Self Employed</b><br>Yes        | <b>Applicant Income</b><br>89000 |
| <b>Coapplicant Income</b><br>78000 | <b>Loan Amount</b><br>50000      |
| <b>Loan Term</b><br>3000           | <b>Credit History</b><br>Good    |
| <b>Property Area</b><br>Rural      |                                  |
| <b>Predict Loan</b>                |                                  |

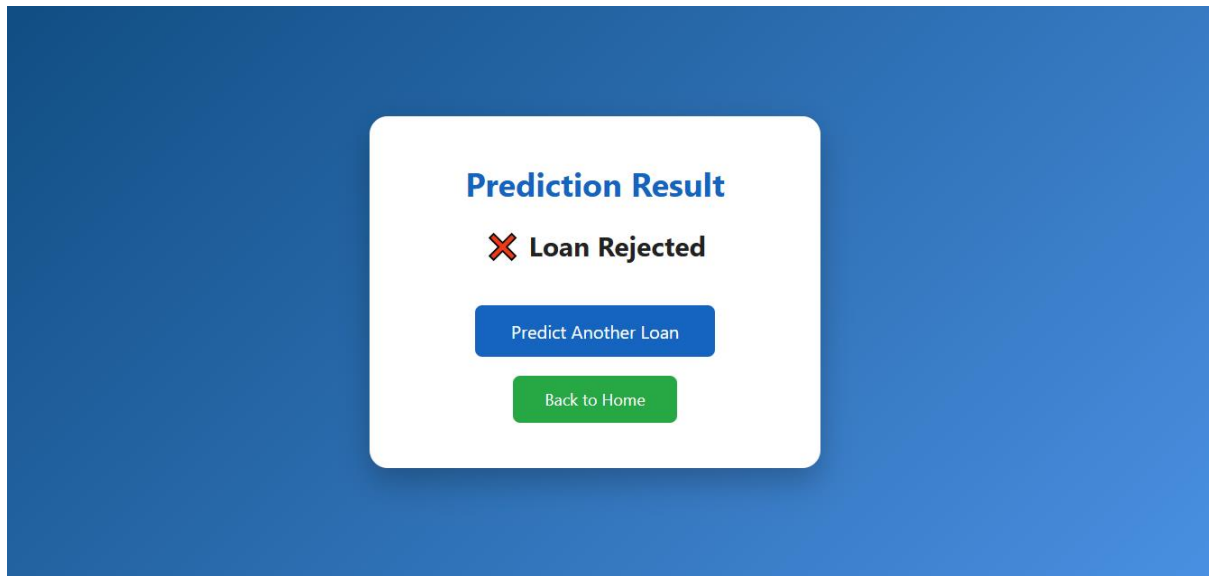
## 5. submit.html

After prediction, Flask displays the final result on this page.

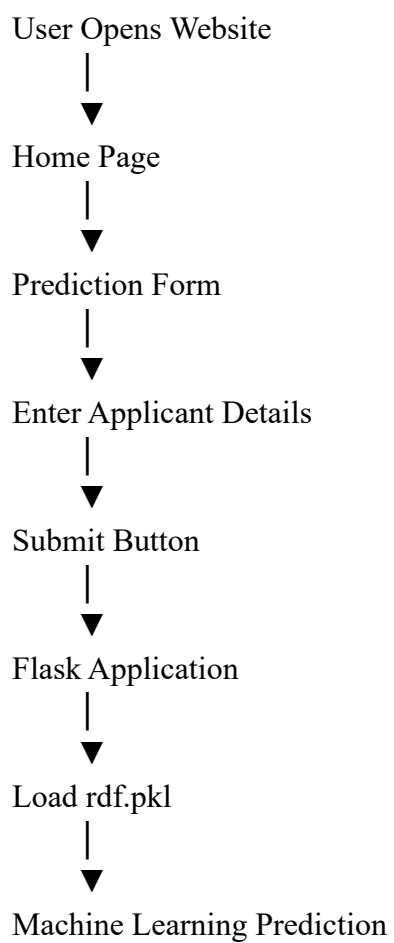
Possible outputs:

✓ Loan Approved

✗ Loan Rejected



### 7.5 Application Workflow





Display Result

### User Input Parameters

The application accepts the following inputs:

| Input               | Description           |
|---------------------|-----------------------|
| Gender              | Male/Female           |
| Married             | Yes/No                |
| Dependents          | Number of Dependents  |
| Education           | Graduate/Not Graduate |
| Self Employed       | Yes/No                |
| Applicant Income    | Monthly Income        |
| Co-applicant Income | Monthly Income        |
| Loan Amount         | Requested Loan Amount |
| Loan Amount Term    | Loan Duration         |
| Credit History      | 0 or 1                |
| Property Area       | Urban/Semiurban/Rural |

These parameters are converted into numerical values before prediction.

### Prediction Process

The prediction process is performed in the following steps.

1. User enters applicant details.
2. Flask receives the form data.
3. Input values are encoded.
4. Machine Learning model receives input.

5. Model predicts loan status.

6. Result is displayed.

### **Application Testing**

The application was tested using different applicant details.

#### **Test Case 1**

Applicant Income: ₹50,000

Credit History: 1

Loan Amount: ₹150

Education: Graduate

#### **Output**

✅ Loan Approved

#### **Test Case 2**

Applicant Income: ₹8,000

Credit History: 0

Loan Amount: ₹300

Education: Not Graduate

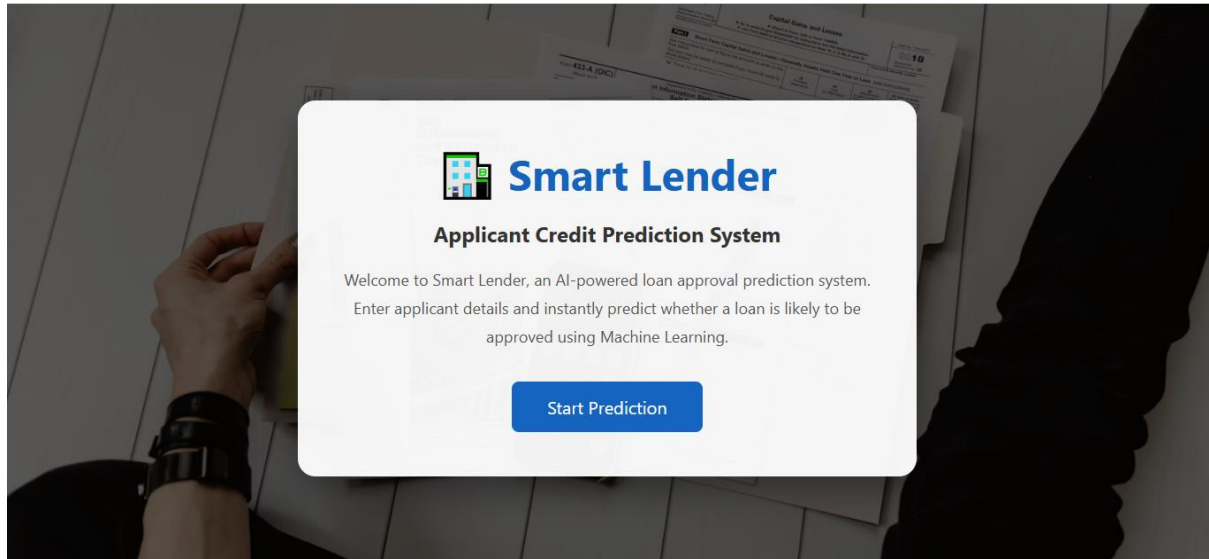
#### **Output**

❌ Loan Rejected

### **Screenshots**

The following screenshots demonstrate the working application.

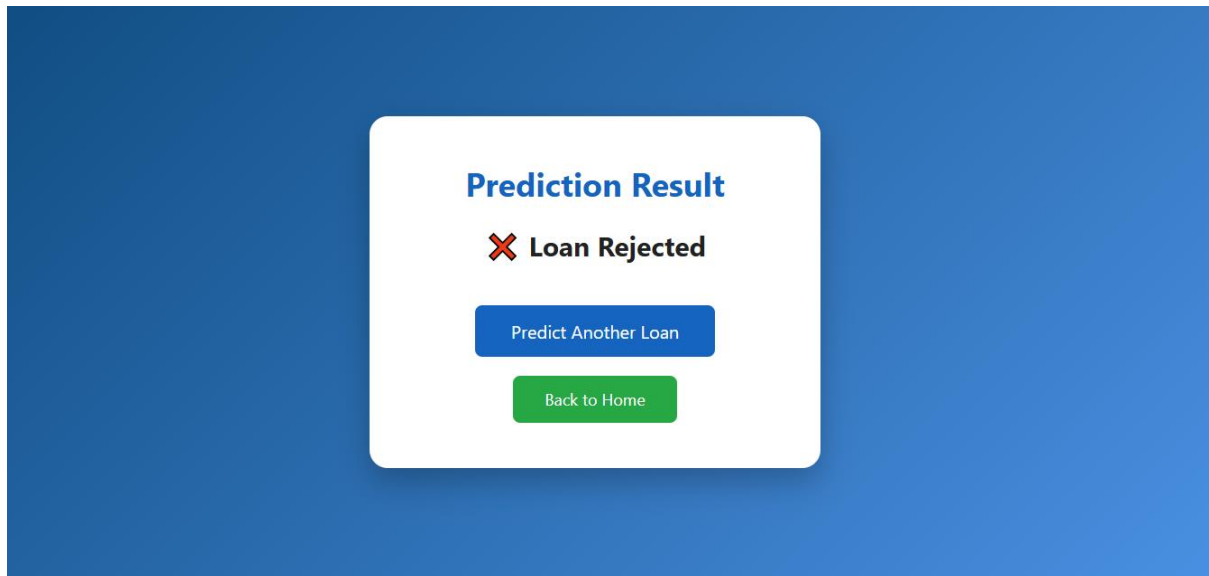
### **Home Page**



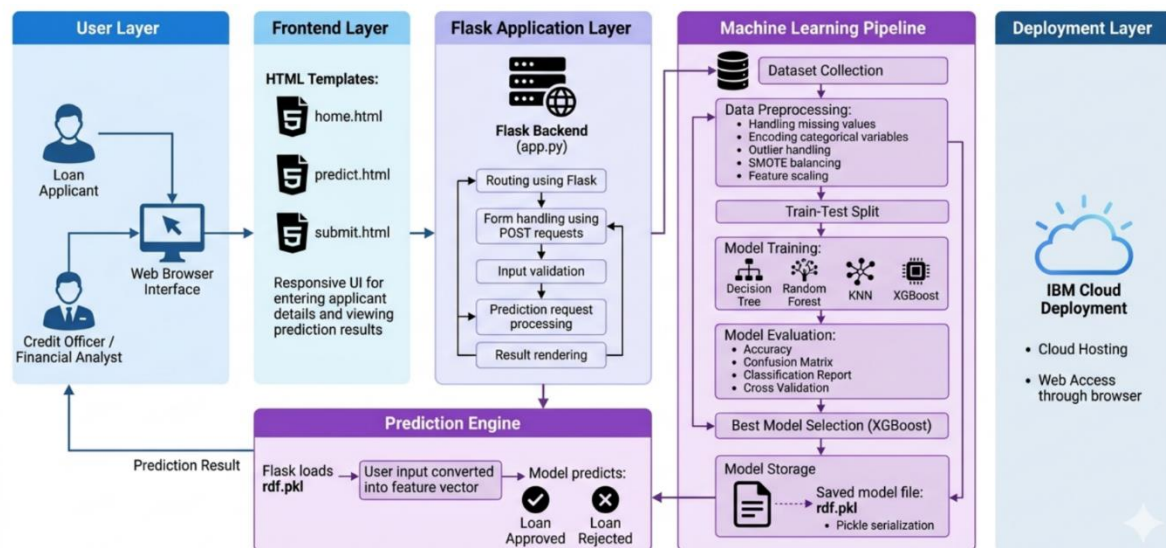
## Prediction Form

|                           |                         |
|---------------------------|-------------------------|
| <b>Gender</b>             | <b>Married</b>          |
| Male                      | Yes                     |
| <b>Dependents</b>         | <b>Education</b>        |
| 2                         | Graduate                |
| <b>Self Employed</b>      | <b>Applicant Income</b> |
| Yes                       | 89000                   |
| <b>Coapplicant Income</b> | <b>Loan Amount</b>      |
| 78000                     | 50000                   |
| <b>Loan Term</b>          | <b>Credit History</b>   |
| 3000                      | Good                    |
| <b>Property Area</b>      |                         |
| Rural                     |                         |
| <b>Predict Loan</b>       |                         |

## Result Page



## Project Workflow



## Advantages of the Application

- Simple user interface.
- Instant loan prediction.
- Fast processing.
- Machine Learning based decision support.

- Reduces manual work.
- Easy to maintain.
- Easy to deploy.

### **Limitations**

Although the application performs well, it has some limitations.

- Uses only historical dataset.
- Accuracy depends on training data quality.
- Does not consider real-time banking policies.
- Limited number of input parameters.
- Cannot replace final bank verification.

### **Future Enhancements**

The Smart Lender project can be improved by adding:

- Integration with real bank databases.
- Aadhaar/PAN verification.
- Credit score APIs.
- Deep Learning models.
- Mobile application.
- Cloud deployment.
- Loan EMI calculator.
- Customer dashboard.
- Multi-language support.
- Real-time analytics.

### **CONCLUSION**

The **Smart Lender – Applicant Creditworthiness Prediction for Loan Approval** project successfully demonstrates the application of Machine Learning in the banking and financial sector.

The project uses historical loan applicant data to train a Random Forest Machine Learning model capable of predicting loan approval decisions. The developed model achieved an accuracy of approximately **81.25%**, indicating reliable prediction performance.

The trained model was integrated with a Flask web application, enabling users to enter applicant details and instantly receive loan approval predictions through a simple web interface.

This project reduces manual effort, improves consistency in decision-making, and demonstrates how Machine Learning can support financial institutions in automating loan approval processes.

Overall, the Smart Lender system provides a practical, scalable, and user-friendly solution for intelligent loan prediction and serves as a strong example of applying Artificial Intelligence and Machine Learning in real-world banking applications.